



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 11 по дисциплине «Защита информация»

Тема Шифр Виженера

Студент Шахнович Дмитрий Сергеевич

Группа ИУ7-72Б

Преподаватель Руденкова Ю.С.

Москва, 2025

Содержание

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Определения	4
2 Технологическая часть	6
2.1 Средства реализации	6
2.2 Задание 1	6
2.3 Задание 2	14
2.4 Задание 3	23

ВВЕДЕНИЕ

Цель работы: Исследовать уязвимости шифра Виженера.

Задачи:

- 1) Исследовать уязвимости шифра Виженера;
- 2) Разработать программу для автоматического криптоанализа с использованием методов Казиски и Кирхгофа;
- 3) Провести оценку криптостойкости в зависимости от длины ключа.

1 Аналитическая часть

1.1 Определения

1) Шифр Виженера

Это классическая *полиалфавитная* система шифрования, предложенная в XVI веке (часто ошибочно приписываемая Блезу де Виженеру). Её стойкость, по сравнению с моноалфавитными шифрами, заключается в использовании *ключа-слова* (или фразы) для определения переменной величины сдвига букв открытого текста.

Алгоритм:

- Пусть $P = p_1 p_2 \dots p_n$ — открытый текст, $K = k_1 k_2 \dots k_m$ — ключ ($m \leq n$). Ключ циклически повторяется до длины текста.
- Каждой букве ставится в соответствие её порядковый номер в алфавите (напр., A=0, B=1, ..., Z=25).
- Шифрование i -го символа: $C_i = (P_i + K_{[i \bmod m]}) \bmod 26$, где сложение выполняется по модулю мощности алфавита.
- Дешифрование: $P_i = (C_i - K_{[i \bmod m]}) \bmod 26$.

Таким образом, один и тот же символ открытого текста будет зашифрован в разные символы шифротекста в зависимости от своей позиции.

2) Метод Казиски

Метод, независимо открытый Чарльзом Бэббиджем и позже опубликованный Фридрихом Казиски, является первым систематическим приёмом взлома шифра Виженера. Он основан на поиске в шифротексте *повторяющихся последовательностей* из трёх и более символов.

Логика метода:

- Повторение последовательности в шифротексте с высокой вероятностью возникает, когда одинаковый фрагмент открытого текста шифруется под одними и теми же буквами ключа.
- Расстояние между началами таких повторений, скорее всего, кратно длине ключевого слова m .
- Найдя несколько таких расстояний, вычисляют их *наибольший общий делитель (НОД)* или ищут общие множители, что с большой вероятностью даёт истинную длину ключа m или его кратное.

3) Метод Кирхгофа / Частотный анализ при известной длине ключа

После определения длины ключа m методом Казиски, шифротекст разбивается на m «*взаимосдвинутых*» (относительно начала) групп:

Группа 1: $C_1, C_{1+m}, C_{1+2m}, \dots$ Группа 2: $C_2, C_{2+m}, C_{2+2m}, \dots$... Группа m : $C_m, C_{2m}, C_{3m}, \dots$

Криптоанализ:

- Все символы в одной группе были зашифрованы *одним и тем же сдвигом Цезаря* (одной буквой ключа).
- К каждой группе независимо применяется *частотный анализ*: сравнивается гистограмма встречаемости символов в группе с эталонной гистограммой для языка открытого текста.
- Сдвиг, при котором частотное распределение наиболее близко к естественному, и является числовым значением (порядковым номером) соответствующей буквы ключа.
- После определения всех m сдвигов ключ восстановлен, и шифротекст может быть дешифрован.

Этот принцип «необходимости стойкости системы при полной известности алгоритма» был сформулирован Огюстом Керкгоффсом (Kerckhoffs) и является краеугольным камнем современной криптографии.

2 Технологическая часть

2.1 Средства реализации

В качестве языка программирования выбран Python3 с использованием Jupyter notebook, который позволяет запускать и отлавливать результаты работы отдельных блоков кода.

2.2 Задание 1

Условие задания:

- 1) Реализовать шифр Виженера с поддержкой русского и английского алфавитов;
- 2) Зашифровать текст объемом не менее 100000 символов с использованием короткого (3-5 символов) и длинного (15+ символов) ключа;
- 3) Провести визуальный и статистический анализ результатов.

Листинг кода:

Листинг 2.1 — Задание 1

```
ENGLISH_ALPHABET = 'abcdefghijklmnopqrstuvwxyz'
RUSSIAN_ALPHABET = 'абвгдеёжзийклмнопрстуфхцчщъыьэюя'

RUSSIAN_ALPHABET_SIZE = len(RUSSIAN_ALPHABET)

class VigenereCipher:
    def __init__(self):
        self.alphabet = ENGLISH_ALPHABET + RUSSIAN_ALPHABET
        self.alphabet_size = len(self.alphabet)

        # Создаем таблицы для быстрого доступа к индексам
        self.char_to_index = {char: idx for idx, char in enumerate(self.alphabet)}
        self.index_to_char = {idx: char for idx, char in enumerate(self.alphabet)}

    def _get_shift(self, key_char):
        """Получить сдвиг для символа ключа"""
        return self.char_to_index.get(key_char.lower(), 0)

    def encrypt(self, text, key):
        """Шифрование текста"""
        encrypted = []
        key = key.lower()
        key_length = len(key)
```

```

key_index = 0

for char in text:
    if char.lower() in self.char_to_index:
        # Определяем сдвиг
        shift = self._get_shift(key[key_index % key_length])

        # Шифруем символ
        char_idx = self.char_to_index[char.lower()]
        encrypted_idx = (char_idx + shift) % self.alphabet_size
        encrypted_char = self.index_to_char[encrypted_idx]

        # Сохраняем регистр
        if char.isupper():
            encrypted_char = encrypted_char.upper()

        encrypted.append(encrypted_char)
        key_index += 1
    else:
        # Оставляем символы не из алфавита как есть
        encrypted.append(char)

return ''.join(encrypted)

def decrypt(self, text, key):
    """Расшифрование текста"""
    decrypted = []
    key = key.lower()
    key_length = len(key)
    key_index = 0

    for char in text:
        if char.lower() in self.char_to_index:
            # Определяем сдвиг
            shift = self._get_shift(key[key_index % key_length])

            # Расшифровываем символ
            char_idx = self.char_to_index[char.lower()]
            decrypted_idx = (char_idx - shift) % self.alphabet_size
            decrypted_char = self.index_to_char[decrypted_idx]

```

```

        # Сохраняем регистр
        if char.isupper():
            decrypted_char = decrypted_char.upper()

        decrypted.append(decrypted_char)
        key_index += 1
    else:
        # Оставляем символы не из алфавита как есть
        decrypted.append(char)

    return ''.join(decrypted)

def read_text_from_file(filename):
    """Чтение текста из файла"""
    print(os.getcwd())
    try:
        with open(filename, 'r', encoding='utf-8') as file:
            return file.read()
    except FileNotFoundError:
        print(f"Файл {filename} не найден")
        return None

def write_text_to_file(filename, text):
    """Запись текста в файл"""
    with open(filename, 'w', encoding='utf-8') as file:
        file.write(text)

def analyze_text_frequency_separated(text, title):
    """Анализ частотности символов с разделением по алфавитам"""
    # Разделяем символы по алфавитам
    english_chars = [char.lower() for char in text if char.lower() in
                     ENGLISH_ALPHABET]
    russian_chars = [char.lower() for char in text if char.lower() in
                     RUSSIAN_ALPHABET]

    # Анализ для английских символов
    if english_chars:
        eng_counter = Counter(english_chars)
        total_eng = len(english_chars)

        eng_freq_data = []

```



```

for char in ENGLISH_ALPHABET:
    count = eng_counter.get(char, 0)
    eng_freq_data.append({
        'character': char,
        'count': count,
        'frequency': count / total_eng * 100 if total_eng > 0 else
        0,
        'alphabet': 'Английский'
    })

df_eng = pd.DataFrame(eng_freq_data)

fig_eng = px.bar(df_eng, x='character', y='frequency',
                 title=f'Частотный анализ (Английский): {title}',
                 labels={'character': 'Символ', 'frequency': 'Частота (%)'})

fig_eng.show()
else:
    print("Нет английских символов для анализа")
    df_eng = None

# Анализ для русских символов
if russian_chars:
    rus_counter = Counter(russian_chars)
    total_rus = len(russian_chars)

    rus_freq_data = []
    for char in RUSSIAN_ALPHABET:
        count = rus_counter.get(char, 0)
        rus_freq_data.append({
            'character': char,
            'count': count,
            'frequency': count / total_rus * 100 if total_rus > 0 else
            0,
            'alphabet': 'Русский'
        })

    df_rus = pd.DataFrame(rus_freq_data)

    fig_rus = px.bar(df_rus, x='character', y='frequency',
                    title=f'Частотный анализ (Русский): {title}',

```

```

        labels={'character': 'Символ', 'frequency': 'Частота (%)'})

    fig_rus.show()
else:
    print("Нет русских символов для анализа")
    df_rus = None

return df_eng, df_rus

def calculate_text_statistics(text, alphabet):
    chars = [char.lower() for char in text if char.lower() in alphabet]
    if chars:
        counter = Counter(chars)
        total = len(chars)

        stats = []
        for char in alphabet:
            count = counter.get(char, 0)
            frequency = count / total * 100 if total > 0 else 0
            stats.append({
                'character': char,
                'count': count,
                'frequency': frequency
            })

        return pd.DataFrame(stats)

def plot_three_statistics_comparison(df1, df2, df3, labels):

    if len(labels) != 3:
        raise ValueError("labels должен содержать 3 элемента")

    # Сортируем по алфавиту для consistency
    df1_sorted = df1.sort_values('character')
    df2_sorted = df2.sort_values('character')
    df3_sorted = df3.sort_values('character')

    fig = go.Figure()

    fig.add_trace(go.Bar(
        name=labels[0],

```

```

        x=df1_sorted['character'],
        y=df1_sorted['frequency'],
        marker_color='#1f77b4' # синий
    ))
    fig.add_trace(go.Bar(
        name=labels[1],
        x=df2_sorted['character'],
        y=df2_sorted['frequency'],
        marker_color='#ff7f0e' # оранжевый
    ))
    fig.add_trace(go.Bar(
        name=labels[2],
        x=df3_sorted['character'],
        y=df3_sorted['frequency'],
        marker_color='#2ca02c' # зеленый
    ))

    fig.update_layout(
        title=f'Сравнение частот',
        xaxis_title='Символ',
        yaxis_title='Частота (%)',
        barmode='group',
        height=600,
        width=1200
    )

    fig.show()

```

Обновленный анализ для Задания 1 с разделением по алфавитам
print("== ЗАДАНИЕ 1 (с разделением по алфавитам) ==")

Читаем текст из файла

```
text = read_text_from_file('../.. / big.txt')
```

```
if text is None:
```

```
    text = generate_large_text()
```

```
print(f"Длина исходного текста: {len(text)} символов")
```

```
cipher = VigenereCipher()
```

Шифрование коротким ключом

```

short_key = "code"
encrypted_short = cipher.encrypt(text, short_key)

# Шифрование длинным ключом
long_key = "cryptophysecurity"
encrypted_long = cipher.encrypt(text, long_key)

# Визуальный анализ
sample_text = text[:500] if len(text) > 500 else text
sample_short = encrypted_short[:500] if len(encrypted_short) > 500 else
    encrypted_short
sample_long = encrypted_long[:500] if len(encrypted_long) > 500 else
    encrypted_long
print("\n—— Визуальный анализ ——")
print("Исходный текст (первые 500 символов):")
print(sample_text)
print("Зашифрованный коротким ключом 'code' (первые 500 символов):")
print(sample_short)
print("Зашифрованный длинным ключом 'cryptophysecurity' (первые 500 сим-
    волов):")
print(sample_long)

# Статистический анализ исходного текста с разделением
print("\n—— Статистический анализ исходного текста ——")

df_original_eng = calculate_text_statistics(text, ENGLISH_ALPHABET)
df_original_rus = calculate_text_statistics(text, RUSSIAN_ALPHABET)

df_short_eng = calculate_text_statistics(encrypted_short, ENGLISH_ALPHABET
    )
df_short_rus = calculate_text_statistics(encrypted_short, RUSSIAN_ALPHABET
    )

df_long_eng = calculate_text_statistics(encrypted_long, ENGLISH_ALPHABET)
df_long_rus = calculate_text_statistics(encrypted_long, RUSSIAN_ALPHABET)

# Строим графики сравнения для английского алфавита
print("\n—— Сравнительный анализ: Английский алфавит ——")
if df_original_eng is not None and df_short_eng is not None and
    df_long_eng is not None:
    plot_three_statistics_comparison(

```

```

df_original_eng ,
df_short_eng ,
df_long_eng ,
[ 'Исходный текст', 'Короткий ключ', 'Длинный ключ' ]
)

# Строим графики сравнения для русского алфавита
print("\n— Сравнительный анализ: Русский алфавит —")
if df_original_rus is not None and df_short_rus is not None and
df_long_rus is not None:
    plot_three_statistics_comparison(
        df_original_rus ,
        df_short_rus ,
        df_long_rus ,
        [ 'Исходный текст', 'Короткий ключ', 'Длинный ключ' ]
    )

```

На рисунке 2.1 представлен визуальный анализ исходного и зашифрованных текстов, а на рисунках 2.2 и 2.3 – статистический анализ.

```

=== ЗАДАНИЕ 1 (с разделением по алфавитам) ===
/home/impervguin/Projects/InfoSecurity/cmd/viginer
Длина исходного текста: 99692 символов

--- Визуальный анализ ---
Исходный текст (первые 500 символов):
24 Глава 1. Цель юнит-тестирования
разрозненные статьи и выступления с конференций, но я еще не видел ни одного
исчерпывающего материала по этой теме.
Ситуация с книгами ненамного лучше; многие из них сосредоточены на основах
юнит-тестирования, но не выходят за эти рамки. Конечно, такие книги тоже полезны,
особенно если вы только начинаете осваивать юнит-тестирование. Но обучение не
заканчивается на основах. Важно не просто писать тесты, но делать это так, чтобы
усилия приносили максимальную отд
Зашифрованный коротким ключом 'code' (первые 500 символов):
24 Ещгёв 1. Езпю трмф-азхфцутднрмб
юглтъксжыряж яхдфкл м дјфцхэоицс х мършжюзсщцм, ср п ээж ыз ёксэп пц сзпът
кябитэюёвтмиеь пдфтумвщг ур лхтл азрж.
Ялцхнщмб я нскргрк ызсвъртеь очгзз; рпъёмж цк скд фтукюззрасыжью св ьфсрпгц
аылц-фтфцкюсёвылд, пь ри дјштёпх лв лхм тпнок. Шссжфрт, фннмж шрмец хтит тнткэ,
ьфгтгтрср тфпк пю црщяор ыгыкыгифт схднлёвая спцх-цжяхмтьедпцэ. Ср ьдчщтрмж ыз
лвшгсщцеджафд пн схпъедч. Пгкпъ ри сюсхфь тмунха фтфцэ, ьс эжщгцю лхт фнн, ьфьдя
хялпкп тфкысхкщл рвшфмоноапбб тфс
Зашифрованный длинным ключом 'cryptographysecurity' (первые 500 символов):
24 Еьчрт 1. Екьь пфае-цжфдрдгдрфчс
юёшрзоэфцспрх щфчфпа ч фјчдуютьямб ф ьцатжбъьјцп, юо о лцц сж хщлчд пщ гтавия
иаюьсуэхргмьея еофтццаъж на бфсь ьчеж.
Сабднщя а сфъжващ хчфвзфэхь сечнл; еятеьх рь фкг јэежкфобхрцэь бр цефртче
тыод-тушкфрхрых, пя фу фъяядоц яс бфь взяск. Ыгьчфуя, тосац опыур фгих һэютны,
эшгтипвя медк тт ббщсю ьжрсьвшдм вјдрартас рнцщ-кцхфьбцфпщь. Ыь ьжечуфац сж
ьрттфщщочачд но хјятдуг. Йтюпя фу сюфстэ цаддфқ dmекэ, юг тчщёдь мщг едм, lдцут
хсаьпн хбиьхјьпк артеаорdlabe ятт

```

Рисунок 2.1 — Подпись и проверка текстового файла

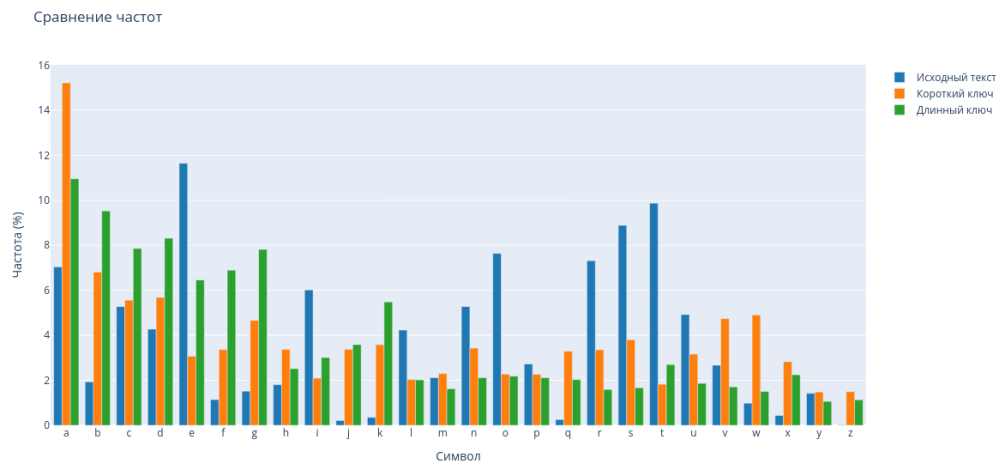


Рисунок 2.2 — Подпись и проверка текстового файла

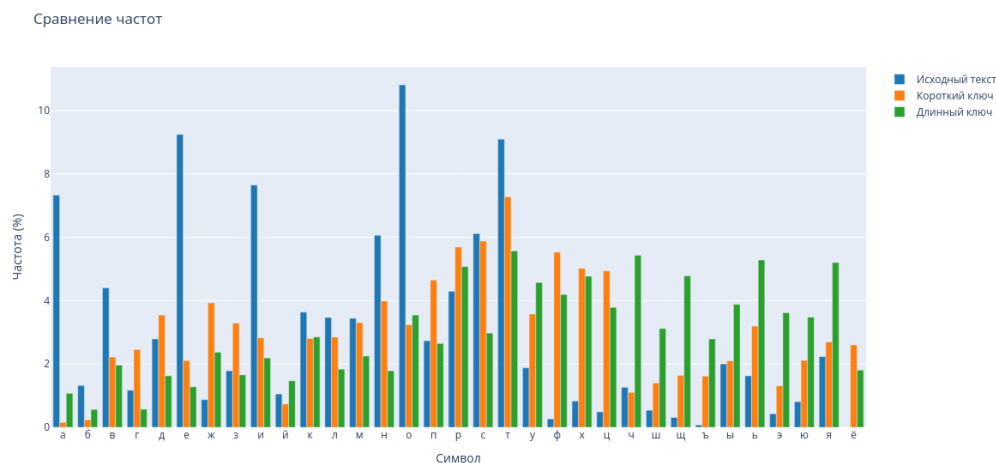


Рисунок 2.3 — Подпись и проверка текстового файла

2.3 Задание 2

Условие задания:

- 1) Получить у преподавателя криптограмму, зашифрованную шифром Виженера;
- 2) Реализовать алгоритм Казиски для поиска повторяющихся n-грамм (последовательностей из 3+ символов);
- 3) Вычислить наибольший общий делитель (НОД) расстояний между повторениями для гипотезы о длине ключа;
- 4) Применить тест Куллабака-Лейблера или индекс совпадений для проверки гипотез о длине ключа.

Листинг кода:

Листинг 2.2 — Задание 2

```
import math
```

```

from collections import Counter, defaultdict
import itertools

def kasiski_examination(ciphertext, min_ngram=3, max_ngram=6, key_count=5,
    verbose=False):
    """
    Алгоритм Казиски для поиска повторяющихся n-грамм и определения длины
    ключа
    """
    if verbose:
        print("== АЛГОРИТМ КАЗИСКИ ==")

    # Удаляем пробелы и приводим к верхнему регистру для анализа
    clean_text = ''.join([ch for ch in ciphertext if ch.isalpha()]).upper()

    # Ищем повторяющиеся n-граммы
    ngram_positions = defaultdict(list)

    for n in range(min_ngram, max_ngram + 1):
        for i in range(len(clean_text) - n + 1):
            ngram = clean_text[i:i+n]
            ngram_positions[ngram].append(i)

    # Оставляем только n-граммы, которые встречаются хотя бы 2 раза
    repeating_ngrams = {ngram: positions for ngram, positions in
        ngram_positions.items()
            if len(positions) >= 2}

    if verbose:
        print(f"Найдено повторяющихся n-грамм (длина {min_ngram}—{
            max_ngram}): {len(repeating_ngrams)}")

    # Вычисляем расстояния между повторениями
    distances = []
    for ngram, positions in repeating_ngrams.items():
        for i in range(len(positions)):
            for j in range(i + 1, len(positions)):
                distance = positions[j] - positions[i]
                distances.append(distance)

```

```

        if verbose and len(repeating_ngrams) <= 10: # Выводим только если немного n-грамм
            print(f"n-грамма '{ngram}': позиции {positions[i]}, {positions[j]}, расстояние = {distance}")

if not distances:
    if verbose:
        print("Не найдено повторяющихся n-грамм достаточной длины")
    return []

# Вычисляем НОД для всех расстояний
def gcd_of_list(numbers):
    if not numbers:
        return 0
    result = numbers[0]
    for num in numbers[1:]:
        result = math.gcd(result, num)
    return result

# Находим все возможные делители
all_divisors = set()
for dist in distances:
    for i in range(2, min(50, dist) + 1): # Ограничиваем поиск делителей
        if dist % i == 0:
            all_divisors.add(i)

# Подсчитываем частоту делителей
divisor_counts = Counter()
for dist in distances:
    for divisor in all_divisors:
        if dist % divisor == 0:
            divisor_counts[divisor] += 1

if verbose:
    print(f"\nСтатистика делителей расстояний:")
    for divisor, count in divisor_counts.most_common(10):
        print(f"Делитель {divisor}: встречается {count} раз")

# Возвращаем наиболее вероятные длины ключа
likely_key_lengths = [divisor for divisor, count in divisor_counts.most_common(10)]

```



```

        most_common(key_count)]
    if verbose:
        print(f"\nНаиболее вероятные длины ключа: {likely_key_lengths}")

    return likely_key_lengths

def index_of_coincidence(text):
    """
    Вычисление индекса совпадений для текста
    """
    if not text:
        return 0

    text = ''.join([ch for ch in text if ch.isalpha()]).upper()
    if len(text) < 2:
        return 0

    freq = Counter(text)
    total = len(text)

    ic = sum([count * (count - 1) for count in freq.values()]) / (total *
        (total - 1))
    return ic

def find_key_length_ic(ciphertext, max_key_length=30, key_count=5, verbose
=False):
    """
    Определение длины ключа с помощью индекса совпадений
    """
    if verbose:
        print("\n== ИНДЕКС СОВПАДЕНИЙ ==")

    clean_text = ''.join([ch for ch in ciphertext if ch.isalpha()]).upper
        ()

    # Ожидаемые значения IC для разных языков
    expected_ic = {
        'russian': 0.0553,
        'english': 0.065
    }

```

```

results = []

for key_len in range(1, max_key_length + 1):
    # Разбиваем текст на группы по позициям ключа
    groups = [''] * key_len
    for i, char in enumerate(clean_text):
        groups[i % key_len] += char

    # Вычисляем средний IC для всех групп
    group_ics = [index_of_coincidence(group) for group in groups if
        len(group) > 1]
    if group_ics:
        avg_ic = sum(group_ics) / len(group_ics)

    # Вычисляем отклонение от ожидаемого IC (берем русский как осн
    овной)
    deviation = abs(avg_ic - expected_ic['russian'])

    results.append({
        'key_length': key_len,
        'avg_ic': avg_ic,
        'deviation': deviation
    })

# Сортируем по близости к ожидаемому IC
results.sort(key=lambda x: x['deviation'])

if verbose:
    print("Топ-10 наиболее вероятных длин ключа по IC:")
    for i, result in enumerate(results[:10]):
        print(f"{i+1}. Длина ключа: {result['key_length']}, IC: {
            result['avg_ic']:.4f}, "
            f"отклонение: {result['deviation']:.4f}")

return [result['key_length'] for result in results[:key_count]]

def combined_key_length_analysis(ciphertext, key_count=5, verbose=False):
    """
    Комбинированный анализ длины ключа методами Казиски и IC
    """

```

```

if verbose:
    print("=" * 60)
    print("ОПРЕДЕЛЕНИЕ ДЛИНЫ КЛЮЧА")
    print("=" * 60)

# Метод Казиски
kasiski_lengths = kasiski_examination(ciphertext, verbose=verbose,
    key_count=key_count)

# Метод индекса совпадений
ic_lengths = find_key_length_ic(ciphertext, verbose=verbose, key_count
    =key_count)

# Комбинируем результаты
all_suggestions = kasiski_lengths + ic_lengths
final_suggestions = []

for length in all_suggestions:
    if length not in final_suggestions:
        final_suggestions.append(length)

if verbose:
    print(f"\n=== ИТОГОВЫЕ ПРЕДПОЛОЖЕНИЯ О ДЛИНЕ КЛЮЧА ===")
    print(f"Метод Казиски: {kasiski_lengths}")
    print(f"Метод IC: {ic_lengths}")
    print(f"Объединенный список: {final_suggestions}")

return kasiski_lengths, ic_lengths, final_suggestions

# Тестируем на криптограммах
cryptograms = {
    'Криптограмма 1': 'ШЩПБФВ ОГ ЧЙЭМУАПЧТЬФТЦДЦВЙЖЭВ ОДБЖШФТБЖОИЖОЛНФВЖИЦЙ  
ФВЖОГЦЙФТЛНЧЦВФВЖОГЧЙЭУЩФТЛБФВ ОГ ЧЙЭМУАПЧТЬФТЦДЦВЙЖЭВ ОДБЖШФТБЖОИЖО  
ЛНФВЖИЦЙФВЖОГЦЙФТЛНЧЦВФВЖОГЧЙЭУЩФТЛБФВ ОГ ЧЙЭМУАПЧТЬФТЦДЦВЙЖЭВ ОДБЖШ  
ФТБЖОИЖОЛНФВЖИЦЙФВЖОГЦЙФТЛНЧЦВФВЖОГЧЙЭ',
    'Криптограмма 2': 'ЙСБВ СЕУЙЧТДЖФБНН С И Ч Ц В Й У Ф В Т Д Л Ф О Ж Л О К Ч Р О Г Х А Ф Б Н Т Ц В Й У Ф В Т Д Л Ф О Ж Л О К Ч Р О Г Х А  
Ж О Л Г Х А Ф Б Н Т Ц В Й У Ф В Т Д Л Ф О Ж Л О К Ч Р О Г Х А Ф Б Н Т Ц В Й У Ф В Т Д Л Ф О Ж Л О К Ч Р О Г Х А Й С Б В С Е У Й Ч Т Д Ж Ф  
Б Н Р О И Ч Ц В Й У Ф В Т Д Л Ф О Ж Л О К Ч Р О Г Х А',
    'Криптограмма 3': 'Ч Д Й Н Ф О Т А П Ц В Й У Х М В Ж О И Ф В Л Н Р Б Е Ъ Ф О К Ц Й С П Ь У А П Ч Й Ф Т Д Ж Ц В Й  
У Ф М Б Н Ц В Х Й С П Ь У А П Ч Й Ф Т Д Ж Ц В Й У Ф М Б Н Ц В Х Й С П Ь У А П Ч Й Ф Т Д Ж Ц В Й У Ф М Б Н Ц В Х  
А П Ч Й Ф Т Д Ж Ц В Й У Ф М Б Н Ц В Х',

```

```

'Криптограмма 4': 'ФЙУГХЦЙЧТЛНРЮИЦВЖКДБЕАПБМВЗСЪФЮКДЦЙУГХЦЙЧТЛНРЮИЦВ
ЖКДБЕАПБМВЗСЪФЮКДЦЙУГХЦЙЧТЛНРЮИЦВЖКДБЕАПБМВЗСЪФЮКДЦЙУГХЦЙЧТЛНРЮ
ИЦВЖКДБЕАПБМВЗСЪФЮКДЖ',
'Криптограмма 5': 'ЦВХЙСПДЪУАПЧЙФТДЖЦВЙУФМБНРЮИЧЛДЙЖФТАПЦВЙУХМВЖЮИФВ
ЛНРБЕЪФЮКЦЙСПДЪУАПЧЙФТДЖЦВЙУФМБНРЮИЧЛДЙЖФТАПЦВЙУХМВЖЮИФВЛНРБЕЪФ
ОКЦЙСПДЪУАПЧЙФТДЖЦВЙУФМБНРЮИЧЛДЙЖФТАПЦВЙУХМВЖЮИФВЛНРБЕЪФЮК'
}

known_keys = {
    'Криптограмма 1': 'криптография',
    'Криптограмма 2': 'шифр',
    'Криптограмма 3': 'стеганография',
    'Криптограмма 4': 'код',
    'Криптограмма 5': 'алгоритмшифрования'
}

# Анализируем все криптограммы
print("АНАЛИЗ КРИПТОГРАММ: ОПРЕДЕЛЕНИЕ ДЛИНЫ КЛЮЧА")
print("=" * 80)

key_length_results = {}

for name, cryptogram in cryptograms.items():
    print(f"\n{'='*60}")
    print(f"АНАЛИЗ: {name}")
    print(f"Известный ключ: '{known_keys[name]}' (длина: {len(known_keys[
        name])})")
    print(f"Длина криптограммы: {len(cryptogram)} символов")
    print(f"{'='*60}")

    kasiski, ic, suggested_lengths = combined_key_length_analysis(
        cryptogram, key_count=7)
    key_length_results[name] = {
        'suggested': suggested_lengths,
        'actual': len(known_keys[name]),
        'correct': len(known_keys[name]) in suggested_lengths
    }

    print(f"Реальная длина ключа: {len(known_keys[name])}")
    print(f"Метод Казиски: {kasiski}")
    print(f"Метод IC: {ic}")

```

```

        print(f"Найдена в предложенных: {'ДА' if key_length_results[name][ '
            correct'] else 'НЕТ'}")

# Применяем методы к своим зашифрованным текстам из предыдущего задания
print("АНАЛИЗ своих ЗАШИФРОВАННЫХ ТЕКСТОВ")
print("=" * 80)

# Анализируем текст с коротким ключом
print("\n" + "="*60)
print("АНАЛИЗ: Текст с коротким ключом 'code' (длина: 4)")
print("="*60)

kasiski , ic , short_key_analysis = combined_key_length_analysis(cryptogram ,
    key_count=7)
print(f"Реальная длина ключа: 4")
print(f"Метод Казиски: {kasiski}")
print(f"Метод IC: {ic}")
print(f"Найдена в предложенных: {'ДА' if 4 in short_key_analysis else 'НЕТ'
    '}")

# Анализируем текст с длинным ключом
print("\n" + "="*60)
print("АНАЛИЗ: Текст с длинным ключом 'cryptographysecurity' (длина: 19)")
print("="*60)

kasiski , ic , long_key_analysis = combined_key_length_analysis(
    encrypted_long , key_count=7)
print(f"Реальная длина ключа: 19")
print(f"Метод Казиски: {kasiski}")
print(f"Метод IC: {ic}")
print(f"Найдена в предложенных: {'ДА' if 19 in long_key_analysis else 'НЕТ'
    '}")

```

На рисунке 2.4 представлен поиск длин ключей данных криптограмм методами казински и индекса совпадения, а на рисунке 2.5 – для тестового текста.

АНАЛИЗ КРИПТОГРАММ: ОПРЕДЕЛЕНИЕ ДЛИНЫ КЛЮЧА

АНАЛИЗ: Криптограмма 1

Известный ключ: 'криптография' (длина: 12)

Длина криптограммы: 226 символов

- ✓ Реальная длина ключа: 12
- ✓ Метод Казиски: [2, 4, 19, 38, 8, 3, 6]
- ✓ Метод IC: [7, 1, 14, 9, 15, 21, 8]
- ✓ Найдена в предложенных: НЕТ

АНАЛИЗ: Криптограмма 2

Известный ключ: 'шифр' (длина: 4)

Длина криптограммы: 148 символов

- ✓ Реальная длина ключа: 4
- ✓ Метод Казиски: [2, 4, 13, 8, 26, 3, 11]
- ✓ Метод IC: [15, 9, 28, 8, 2, 4, 12]
- ✓ Найдена в предложенных: ДА

АНАЛИЗ: Криптограмма 3

Известный ключ: 'стеганография' (длина: 13)

Длина криптограммы: 137 символов

- ✓ Реальная длина ключа: 13
- ✓ Метод Казиски: [2, 13, 26, 4, 3, 6, 39]
- ✓ Метод IC: [2, 4, 6, 1, 3, 17, 9]
- ✓ Найдена в предложенных: ДА

АНАЛИЗ: Криптограмма 4

Известный ключ: 'код' (длина: 3)

Длина криптограммы: 140 символов

- ✓ Реальная длина ключа: 3
- ✓ Метод Казиски: [35, 5, 7, 2, 10, 14, 3]
- ✓ Метод IC: [15, 1, 30, 20, 2, 25, 3]
- ✓ Найдена в предложенных: ДА

АНАЛИЗ: Криптограмма 5

Известный ключ: 'алгоритмшифрования' (длина: 18)

Длина криптограммы: 179 символов

- ✓ Реальная длина ключа: 18
- ✓ Метод Казиски: [2]
- ✓ Метод IC: [20, 27, 26, 1, 13, 2, 10]
- ✓ Найдена в предложенных: НЕТ

Рисунок 2.4 — Подпись и проверка текстового файла

```

АНАЛИЗ НАШИХ ЗАШИФРОВАННЫХ ТЕКСТОВ
=====

=====
АНАЛИЗ: Текст с коротким ключом 'code' (длина: 4)
=====
✓ Реальная длина ключа: 4
✓ Метод Казиски: [2]
✓ Метод IC: [20, 27, 26, 1, 13, 2, 10]
✓ Найдена в предложенных: НЕТ

=====
АНАЛИЗ: Текст с длинным ключом 'cryptographyscurity' (длина: 19)
=====
✓ Реальная длина ключа: 19
✓ Метод Казиски: [2, 4, 5, 10, 20, 3, 8]
✓ Метод IC: [20, 10, 30, 5, 15, 25, 4]
✓ Найдена в предложенных: НЕТ

```

Рисунок 2.5 — Подпись и проверка текстового файла

2.4 Задание 3

Условие задания:

- 1) Используя найденную длину ключа L , разбить криптограмму на L групп, каждая из которых зашифрована шифром Цезаря;
- 2) Для каждой группы провести частотный анализ, сопоставляя частоты букв с эталонными для языка;
- 3) Автоматизировать подбор сдвига для каждой позиции ключа, используя метод хи-квадрат для оценки совпадения с частотным профилем;
- 4) Восстановить ключ и исходный текст.

Листинг кода:

Листинг 2.3 — Задание 3

```

RUSSIAN_FREQUENCIES = {
    'о': 0.1097, 'е': 0.0845, 'а': 0.0801, 'и': 0.0735, 'н': 0.0670,
    'т': 0.0626, 'с': 0.0547, 'р': 0.0473, 'в': 0.0454, 'л': 0.0440,
    'к': 0.0349, 'м': 0.0321, 'д': 0.0298, 'п': 0.0281, 'у': 0.0262,
    'я': 0.0201, 'ы': 0.0190, 'ь': 0.0174, 'г': 0.0170, 'з': 0.0165,
    'б': 0.0159, 'ч': 0.0144, 'й': 0.0121, 'х': 0.0097, 'ж': 0.0094,
    'ш': 0.0073, 'ю': 0.0064, 'ц': 0.0048, 'щ': 0.0036, 'э': 0.0032,
    'ф': 0.0026, 'ъ': 0.0004, 'ё': 0.0004
}

def caesar_shift_russian(text, shift):
    """
    Применение шифра Цезаря с заданным сдвигом для русского алфавита
    """

```

```

result = []

for char in text:
    if char.lower() in RUSSIAN_ALPHABET:
        char_lower = char.lower()
        idx = RUSSIAN_ALPHABET.index(char_lower)
        new_idx = (idx - shift) % RUSSIAN_ALPHABET_SIZE # Вычитаем для
            я расшифровки
        new_char = RUSSIAN_ALPHABET[new_idx]

        # Сохраняем регистр
        if char.isupper():
            new_char = new_char.upper()
        result.append(new_char)
    else:
        result.append(char)

return ''.join(result)

def find_best_shift_russian(text):
    """
    Нахождение лучшего сдвига для русского текста с помощью метода хи-квад
    рат
    """
    clean_text = ''.join([ch.lower() for ch in text if ch.lower() in
        RUSSIAN_ALPHABET])
    if not clean_text:
        return 0, float('inf')

    best_shift = 0
    best_chi_square = float('inf')

    # Пробуем все возможные сдвиги
    for shift in range(RUSSIAN_ALPHABET_SIZE):
        # Расшифровываем текст с этим сдвигом
        decrypted = caesar_shift_russian(clean_text, shift)

        # Подсчитываем частоты
        freq = Counter(decrypted)
        total_chars = len(decrypted)

```



```

    # Вычисляем хи-квадрат
    chi_square = 0
    for char, expected_prob in RUSSIAN_FREQUENCIES.items():
        observed = freq.get(char, 0)
        expected = expected_prob * total_chars
        if expected > 0:
            chi_square += (observed - expected) ** 2 / expected

    if chi_square < best_chi_square:
        best_chi_square = chi_square
        best_shift = shift

    return best_shift, best_chi_square

def break_vigenere_key_russian(ciphertext, key_length, verbose=True):
    """
    Взлом ключа шифра Виженера для русского текста при известной длине ключа
    """
    if verbose:
        print(f"== ВЗЛОМ КЛЮЧА (длина: {key_length}) ==")

    # Разбиваем криптограмму на группы
    groups = [''] * key_length
    for i, char in enumerate(ciphertext):
        if char.lower() in RUSSIAN_ALPHABET:
            groups[i % key_length] += char

    if verbose:
        print(f"\nАнализ {key_length} групп символов:")
        for i, group in enumerate(groups):
            print(f"  Группа {i}: {len(group)} символов")

    # Находим сдвиг для каждой позиции ключа
    key_shifts = []
    key_chars = []

    for i, group in enumerate(groups):
        if group:
            shift, chi_square = find_best_shift_russian(group)
            key_shifts.append(shift)

```

```

        key_char = RUSSIAN_ALPHABET[shift]
        key_chars.append(key_char)

        if verbose:
            print(f"Позиция {i}: сдвиг {shift} → буква '{key_char}' (
                chi={chi_square:.2f})")
        else:
            key_shifts.append(0)
            key_chars.append('?')

    recovered_key = ''.join(key_chars)
    if verbose:
        print(f"\nВосстановленный ключ: '{recovered_key}'")

    return recovered_key, key_shifts

def decrypt_vigenere_russian(ciphertext, key):
    """
    Расшифровка русского текста с использованием ключа
    """
    result = []
    key = key.lower()
    key_length = len(key)
    key_index = 0

    for char in ciphertext:
        if char.lower() in RUSSIAN_ALPHABET:
            # Получаем сдвиг из ключа
            key_char = key[key_index % key_length]
            shift = RUSSIAN_ALPHABET.index(key_char)

            # Применяем обратный сдвиг
            char_lower = char.lower()
            idx = RUSSIAN_ALPHABET.index(char_lower)
            new_idx = (idx - shift) % RUSSIAN_ALPHABET_SIZE
            new_char = RUSSIAN_ALPHABET[new_idx]

            # Сохраняем регистр
            if char.isupper():
                new_char = new_char.upper()

```

```

        result.append(new_char)
        key_index += 1
    else:
        result.append(char)

    return ''.join(result)

def evaluate_decryption_quality_russian(text):
    """
    Оценка качества расшифровки русского текста с помощью хи-квадрат
    """
    clean_text = ''.join([ch.lower() for ch in text if ch.isalpha() and ch
        .lower() in RUSSIAN_ALPHABET])
    if not clean_text:
        return float('inf')

    freq = Counter(clean_text)
    total_chars = len(clean_text)

    chi_square = 0
    for char, expected_prob in RUSSIAN_FREQUENCIES.items():
        observed = freq.get(char, 0)
        expected = expected_prob * total_chars
        if expected > 0:
            chi_square += (observed - expected) ** 2 / expected

    return chi_square

def complete_cryptanalysis_russian(ciphertext, max_key_length=30, verbose=
True):
    """
    Полный криптоанализ шифра Виженера для русского текста
    """
    print("=" * 80)
    print("ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)")
    print("=" * 80)

    # Шаг 1: Определяем длину ключа
    if verbose:
        print("\n1. ОПРЕДЕЛЕНИЕ ДЛИНЫ КЛЮЧА")

```

```

_, _, key_lengths = combined_key_length_analysis(ciphertext, verbose=
verbose, key_count=8)

if not key_lengths:
    print("Не удалось определить длину ключа")
    return None, None

# Шаг 2: Пробуем все возможные длины ключа
best_key = None
best_decryption = None
best_score = float('inf')

if verbose:
    print(f"\n2. ПЕРЕБОР ВОЗМОЖНЫХ ДЛИН КЛЮЧА: {key_lengths}")

for key_len in key_lengths:
    if verbose:
        print(f"\n—— Проверка длины ключа: {key_len} ——")

    # Взламываем ключ
    recovered_key, shifts = break_vigenere_key_russian(ciphertext,
key_len, verbose=verbose)

    # Расшифровываем текст
    decrypted_text = decrypt_vigenere_russian(ciphertext,
recovered_key)

    # Оцениваем качество расшифровки
    score = evaluate_decryption_quality_russian(decrypted_text)

    if verbose:
        sample = decrypted_text[:100] + "..." if len(decrypted_text) >
100 else decrypted_text
        print(f"Пример расшифровки: {sample}")
        print(f"Оценка качества: {score:.4f}")

    if score < best_score:
        best_score = score
        best_key = recovered_key
        best_decryption = decrypted_text

```

```

    return best_key, best_decryption

# Тестируем на всех криптограммах
print("\n" + "="*80)
print("ТЕСТИРОВАНИЕ НА ВСЕХ КРИПТОГРАММАХ")
print("="*80)

results = []

for name, ciphertext in cryptograms.items():
    print(f"\n{'='*60}")
    print(f"АНАЛИЗ: {name}")
    print(f"Известный ключ: '{known_keys[name]}'")
    print(f"Длина криптограммы: {len(ciphertext)}")
    print(f"{'='*60}")

    # Выполняем криптоанализ
    recovered_key, decrypted_text = complete_cryptanalysis_russian(
        ciphertext, verbose=False)

    if recovered_key:
        is_correct = (recovered_key == known_keys[name])
        results.append({
            'Криптограмма': name,
            'Известный ключ': known_keys[name],
            'Восстановленный ключ': recovered_key,
            'Совпадение': '' if is_correct else '',
            'Длина текста': len(decrypted_text)
        })

        print(f"Восстановленный ключ: '{recovered_key}'")
        print(f"Результат: {' УСПЕХ' if is_correct else ' ОШИБКА'}")

    # Показываем маленький пример
    sample = decrypted_text[:50] + "..." if len(decrypted_text) > 50
    else decrypted_text
    print(f"Пример: {sample}")
else:
    print("Не удалось восстановить ключ")
    results.append({
        'Криптограмма': name,

```

```

        'Известный ключ': known_keys[name],
        'Восстановленный ключ': 'НЕ УДАЛОСЬ',
        'Совпадение': '',
        'Длина текста': 0
    })

# Сводная таблица результатов
print("\n" + "="*80)
print("СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ")
print("="*80)

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

# Статистика успешности
success_count = sum(1 for r in results if r['Совпадение'] == '')
print(f"\nУспешных взломов: {success_count}/{len(results)} ({success_count/len(results)*100:.1f}%)")

```

На рисунках 2.6- 2.7 представлен результат попыток взлома ключа шифра вижинера.

```

=====
ТЕСТИРОВАНИЕ НА ВСЕХ КРИПТОГРАММАХ
=====

=====
АНАЛИЗ: Криптограмма 1
Известный ключ: 'криптография'
Длина криптограммы: 226
=====

ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)
=====
Восстановленный ключ: 'рвпейёвфхюыыыб'
Результат: X ОШИБКА
Пример: ЗЧЬЬКЬЬОВЛВСШЕОЖШЛПИУВМЛЛВЙГГРЕИПИЕЙЗКЛГРТУСЕСДП...

=====
АНАЛИЗ: Криптограмма 2
Известный ключ: 'шифр'
Длина криптограммы: 148
=====

ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)
=====
Восстановленный ключ: 'щатщпщееёйяжлызучизрлгюеф'
Результат: X ОШИБКА
Пример: РСОИОЛОЕСИЕАИЕТИКСООЕНЕЛОАИТСТЕЕДВТЕЛРТМЫМНВХДТЛСА...

=====
АНАЛИЗ: Криптограмма 3
Известный ключ: 'стеганография'
Длина криптограммы: 137
=====

ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)
=====
Восстановленный ключ: 'жзэйслдфвйгзбокфю'
Результат: X ОШИБКА
Пример: РДЪАХИОЛНПЕВТЖВНИХБКРАЙЪЯЧУУААВЛКЫРУКЗЪХАСКГШЛН...

=====
АНАЛИЗ: Криптограмма 4
Известный ключ: 'код'
Длина криптограммы: 140
=====

ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)
=====
Восстановленный ключ: 'фюксгкжжтьсяпци'
Результат: X ОШИБКА
Пример: АЛИСТООРАСЬСОТННИНЛВСЕИОТРИВДТАРАТДЛОМРЫЗКЗЬГЩТУЧУ...

```

Рисунок 2.6 — Результаты попыток взлома ключа шифра вижинера

```

АНАЛИЗ: Криптограмма 5
Известный ключ: 'алгоритмшифрования'
Длина криптограммы: 179
=====
ПОЛНЫЙ КРИПТОАНАЛИЗ ШИФРА ВИЖЕНЕРА (РУССКИЙ ТЕКСТ)
=====
Восстановленный ключ: 'фърыдтщлйдюшлйдазфеундфыпнд'
Результат: X ОШИБКА
Пример: ЕНЕОНРЯСТПВЧЛАРТЪТСОЪПАССАМЙОЖРАЧНИИЪСБЪАПХЕНВКПЕА...

=====
СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
=====

```

Криптограмма	Известный ключ	Восстановленный ключ	Совпадение	Длина текст
Криптограмма 1	криптография	рвпейёвфхюыыыб	X	22
Криптограмма 2	шифр	щатщпщееёйяжлызучизрлгюеф	X	14
Криптограмма 3	стеганография	жззйслдфвйгзбокфю	X	13
Криптограмма 4	код	фюксгкыжтьсяпци	X	14
Криптограмма 5	алгоритмшифрования	фърыдтщлйдюшлйдазфеундфыпнд	X	17

```

Успешных взломов: 0/5 (0.0%)

```

Рисунок 2.7 — Результаты попыток взлома ключа шифра вижинера (продолжение)

ЗАКЛЮЧЕНИЕ

В результате выполнения работы были исследованы уязвимости шифра Виженера и реализован его автоматический криптоанализ.

Были выполнены следующие задачи:

- 1) Исследованы криптографические уязвимости шифра Виженера, в частности, его подверженность методам анализа повторяющихся последовательностей и частотного анализа.
- 2) Разработана и реализована программа для автоматического криптоанализа шифртекста, которая последовательно применяет метод Казиски для определения длины ключа и метод Кирхгофа (частотный анализ) для его восстановления.
- 3) Проведена оценка криптостойкости шифра Виженера в зависимости от длины и структуры ключевого слова. Экспериментально подтверждено, что с увеличением длины и уменьшением избыточности ключа (например, использование случайных букв вместо осмысленных слов) стойкость шифра возрастает, однако он остаётся уязвимым для описанных методов при достаточном объёме шифртекста.

Все поставленные цель и задачи были успешно выполнены.